



Database After Graduation

Week 1

**By Narges Yarahmadi
Feb 2026**

Table of Contents

SECTION 1:	4
BEFORE QUERYING: HOW TO THINK ABOUT DATABASE MODELING	5
STEP 1: START FROM THE REAL WORLD, NOT FROM TABLES	5
STEP 2: IDENTIFY ENTITIES	5
STEP 3: DEFINE ATTRIBUTES FOR EACH ENTITY	6
STEP 4: DEFINE RELATIONSHIPS BETWEEN ENTITIES	6
STEP 5: CHOOSE PRIMARY KEYS CAREFULLY	7
STEP 6: NORMALIZE THE MODEL FIRST	7
STEP 7: THINK ABOUT QUESTIONS THE DATABASE MUST ANSWER	8
STEP 8: HOW MODELING AFFECTS QUERYING (BIG PICTURE)	8
BRIDGE TO SQL REVIEW	9
SECTION 2: SQL REVIEW – QUERYING A WELL-MODELED DATABASE	9
THE GOAL: FROM DATA TO INSIGHT	9
CORE SQL REVIEW IN A MODELED CONTEXT	10
<i>A. The SELECT & FROM Foundation: Choosing Your Tables</i>	10
<i>B. JOIN: Navigating Relationships</i>	10
<i>C. WHERE: Filtering Results</i>	11
<i>D. GROUP BY & Aggregate Functions: Summarizing Data</i>	11
<i>E. HAVING: Filtering Groups</i>	11
<i>F. ORDER BY & LIMIT: Delivering Refined Results</i>	11
BEST PRACTICES FOR QUERYING A WELL-MODELED DATABASE	12
COMMON PITFALLS & HOW A GOOD MODEL SAVES YOU	12
DEMONSTRATION: FROM QUESTION TO QUERY	13
SECTION 3 :NORMALIZATION IN DATABASE DESIGN	13
WHY NORMALIZATION EXISTS	14
CORE IDEA: ONE FACT, ONE PLACE	14
FUNCTIONAL DEPENDENCY (INTUITION, NOT MATH)	14
FIRST NORMAL FORM (1NF): ATOMIC VALUES	15
SECOND NORMAL FORM (2NF): NO PARTIAL DEPENDENCY	15
THIRD NORMAL FORM (3NF): NO TRANSITIVE DEPENDENCY	16
WHAT NORMALIZATION GIVES YOU	16
NORMALIZATION IS A STARTING POINT, NOT A RELIGION	17
KEY TAKEAWAY	17
NORMALIZATION VS DENORMALIZATION	17
DENORMALIZATION IN REAL-WORLD SYSTEMS	18
WHY DENORMALIZATION EXISTS	18
THE CORE TRADE-OFF	18
EXAMPLE 1: NORMALIZED VS DENORMALIZED ORDERS	19
EXAMPLE 2: DUPLICATING DESCRIPTIVE FIELDS	19
EXAMPLE 3: PRE-COMPUTED AGGREGATES	20
KEEPING DENORMALIZED DATA IN SYNC	20
WHEN DENORMALIZATION MAKES SENSE	21
WHEN DENORMALIZATION IS DANGEROUS	21
SIDE-BY-SIDE DIAGRAM OF NORMALIZED VS DENORMALIZED DESIGN	21
EXERCISE: “WHAT WOULD YOU DENORMALIZE AND WHY?”	23
KEY TAKEAWAYS	24
SECTION 4: DESIGNING FOR READ-HEAVY VS WRITE-HEAVY SYSTEMS	25

EXAMPLES OF READ-HEAVY AND WRITE-HEAVY SYSTEMS	25
READ-HEAVY SYSTEMS	25
EXAMPLE 1: “MY ORDERS” PAGE IN A FOOD DELIVERY APP	25
EXAMPLE 2: RESTAURANT DASHBOARD	26
EXAMPLE 3: USER PROFILE PAGES (SOCIAL / E-COMMERCE SYSTEMS)	26
WRITE-HEAVY SYSTEMS	27
EXAMPLE 4: LOGGING SYSTEMS.....	27
EXAMPLE 5: PAYMENT SYSTEMS.....	27
EXAMPLE 6: EVENT INGESTION AND TELEMETRY SYSTEMS.....	28
QUICK COMPARISON	28
KEY TAKEAWAY	29

Section 1: Data Modeling for Real-World Systems

Data modeling is the foundation of every reliable data system. Before performance tuning, before scaling, before choosing a database technology, there is always a model that represents how the real world is translated into structured data. A good data model reduces complexity, prevents inconsistencies, and makes systems easier to reason about as they grow.

In real-world systems, data modeling is not only a theoretical exercise. It is a practical decision-making process shaped by business requirements, access patterns, and long-term maintenance concerns.

Entities, Relationships, and Schema Design

At its core, data modeling starts with identifying **entities**. An entity represents a real-world object or concept that the system needs to store information about, such as a user, order, product, or transaction. Each entity is described by attributes, which become columns in a table or fields in a document.

Relationships describe how entities are connected. Common relationship types include one-to-one, one-to-many, and many-to-many. Choosing the correct relationship type is critical, because it directly affects data integrity and query complexity. For example, modeling a one-to-many relationship incorrectly as many-to-many can introduce unnecessary join tables and complicate queries.

Schema design is the process of turning entities and relationships into a structured representation that a database can enforce. A well-designed schema reflects business rules clearly. Constraints such as primary keys, foreign keys, and uniqueness are not optional details. They are part of the model itself and act as guardrails that protect data correctness over time.

Query-Driven Data Modeling

In real systems, data is rarely modeled in isolation from how it will be used. **Query-driven data modeling** starts by asking a simple but powerful question: *What questions will this system need to answer?*

Instead of designing tables first and hoping queries work later, we analyze expected access patterns early. For example, if the system frequently needs to retrieve recent orders for a user, that access path should be easy and efficient in the schema. If analytics queries dominate, the model may prioritize aggregation and scan efficiency.

This approach does not mean ignoring correctness or structure. It means acknowledging that reads and writes have costs, and a model should reflect the most important operations the system

performs. Query-driven modeling is especially important in distributed systems and data platforms where inefficient queries become expensive at scale.

Before Querying: How to Think About Database Modeling

Before writing queries, we need to understand what we are querying. SQL is not just a syntax for fetching data. It is a language for asking questions about a **model of the real world**.

A database model is the bridge between:

- Real-world concepts (users, orders, payments)
- And structured data (tables, rows, keys)

If the model is unclear, queries become confusing, slow, and error-prone. If the model is clear, queries become almost self-explanatory.

Step 1: Start From the Real World, Not From Tables

Good database modeling always starts with questions like:

- What things exist in this system?
- What do we need to remember about them?
- How do they relate to each other?

For example, in a food delivery system:

- Users place orders
- Orders belong to restaurants
- Orders contain multiple items
- Payments are made for orders

At this stage, we do not think about SQL. We think about **concepts**.

Step 2: Identify Entities

An **entity** represents a distinct concept that has its own identity.

Examples of entities:

- User
- Order
- Restaurant
- Menu Item
- Payment

A useful rule of thumb:

If something has its own lifecycle and can exist independently, it is probably an entity.

Each entity will usually become a table.

Step 3: Define Attributes for Each Entity

Once we identify entities, we decide what information we need to store about each one.

For example:

- User → name, email, created_at
- Order → status, created_at
- Restaurant → name, city
- Payment → amount, method, paid_at

These attributes become columns.

Important idea:

Columns should describe the entity itself, not related entities.

If a column describes something else, it likely belongs in another table.

Step 4: Define Relationships Between Entities

Entities rarely exist in isolation.

We must understand:

- One-to-one relationships
- One-to-many relationships
- Many-to-many relationships

Examples:

- One user → many orders
- One restaurant → many menu items
- One order → many order items

Relationships explain **why JOINS exist**.

Without relationships, JOIN has no meaning.

Foreign keys are how relationships are expressed in a relational database.

Step 5: Choose Primary Keys Carefully

Every table needs a primary key.

A primary key:

- Uniquely identifies a row
- Never changes
- Has no business meaning outside identification

Examples:

- user_id
- order_id
- restaurant_id

Primary keys are the anchors of joins and grouping operations later.

Step 6: Normalize the Model First

Before worrying about performance, we normalize.

Normalization ensures:

- Each fact is stored once
- Data is consistent
- Updates are safe

A normalized model gives us:

- Clear table boundaries

- Predictable joins
- Correct aggregation results

This is why normalization comes **before** querying.

Step 7: Think About Questions the Database Must Answer

Now we shift perspective.

Instead of asking:

“How do I write SQL?”

We ask:

“What questions will this system need to answer?”

Examples:

- Show a user’s last 10 orders
- Calculate total revenue per restaurant
- Count how many items were sold today

These questions directly map to:

- SELECT → what data
- JOIN → how tables relate
- WHERE → which rows
- GROUP BY → how data is aggregated

If the model is correct, these queries feel natural.

Step 8: How Modeling Affects Querying (Big Picture)

Bad modeling leads to:

- Confusing JOINS
- Incorrect aggregates
- Duplicate rows
- Complex queries

Good modeling leads to:

- Simple SELECT statements
 - Meaningful GROUP BY clauses
 - Predictable results
-

Bridge to SQL Review

Now that we understand how data is modeled,
we can learn how to **ask questions** about that model using SQL.

Or more explicitly:

SELECT asks what we want.

JOIN follows relationships.

GROUP BY summarizes entities.

Section 2: SQL Review – Querying a Well-Modeled Database

The Goal: From Data to Insight

The core purpose of a relational database is not merely to *store* data, but to *answer questions* with it. SQL (Structured Query Language) is our tool for asking those questions. However, the **efficiency, clarity, and reliability** of our answers depend entirely on the **quality of the underlying data model**.

A **well-modeled database** provides us with:

- **Clarity:** Tables represent clear, single-purpose entities (e.g., Customers, Products, Orders).
- **Integrity:** Relationships are explicitly defined through foreign keys, preventing invalid data.
- **Efficiency:** Data is stored without unnecessary redundancy (normalized), reducing update anomalies and saving space.
- **A Roadmap for Queries:** The schema itself acts as a guide for how to join tables together logically.

Key Principle: Writing SQL against a well-designed database feels like navigating a well-labeled map. Writing it against a poorly designed one feels like searching a messy attic in the dark.

Core SQL Review in a Modeled Context

Let's frame our standard SQL clauses within the context of a classic, well-modeled schema: an **e-commerce system** with Customers, Orders, Order_Details, and Products.

A. The SELECT & FROM Foundation: Choosing Your Tables

The FROM clause is your starting point on the data model map. You begin at the table that holds your primary perspective.

```
sql
-- Single-Table Query: Start simple.
SELECT customer_name, email
FROM Customers;
```

In a well-modeled DB, you will almost always need multiple tables. This is a *feature*, not a bug.

B. JOIN: Navigating Relationships

Joins follow the "paths" created by foreign keys.

- **INNER JOIN:** The workhorse. Retrieve only records where a match exists in **both** tables.

```
sql

-- "Get all orders along with the customer's name for each."
SELECT Orders.order_id, Orders.order_date, Customers.customer_name
FROM Orders
INNER JOIN Customers ON Orders.customer_id = Customers.customer_id;
```

Visually, this connects the "Orders" table to the "Customers" table via the customer_id highway.

- **LEFT JOIN:** Preserve all records from the first (left) table, even if there's no match.

```
sql

-- "List all customers and any orders they might have (including customers who haven't ordered)."
SELECT Customers.customer_name, Orders.order_id
FROM Customers
LEFT JOIN Orders ON Customers.customer_id = Orders.customer_id;
```

- **Multi-Table JOIN:** Traversing the web of relationships.

```
sql

-- "Get the details of each order, including the product name and price."
SELECT O.order_id, C.customer_name, P.product_name, OD.quantity, OD.unit_price
FROM Orders O
INNER JOIN Customers C ON O.customer_id = C.customer_id
```

```
INNER JOIN Order_Details OD ON O.order_id = OD.order_id
INNER JOIN Products P ON OD.product_id = P.product_id;
```

This query walks across three explicit relationships in our model.

C. WHERE: Filtering Results

Apply conditions **after** joins are established. Use it to restrict rows based on column values.

```
sql
-- "Find all orders in January 2023 for customers from New York."
SELECT ...
FROM Orders O
INNER JOIN Customers C ON O.customer_id = C.customer_id
WHERE O.order_date >= '2023-01-01'
      AND O.order_date < '2023-02-01'
      AND C.state = 'NY';
```

D. GROUP BY & Aggregate Functions: Summarizing Data

This is where the power of a relational model shines. We can summarize data across dimensions.

```
sql
-- "Calculate the total revenue generated by each customer."
SELECT C.customer_id, C.customer_name,
       SUM(OD.quantity * OD.unit_price) AS total_revenue
FROM Customers C
LEFT JOIN Orders O ON C.customer_id = O.customer_id
LEFT JOIN Order_Details OD ON O.order_id = OD.order_id
GROUP BY C.customer_id, C.customer_name;
```

Important: In a GROUP BY, all non-aggregated columns in the SELECT must appear in the GROUP BY clause. The primary key is a safe choice.

E. HAVING: Filtering Groups

The WHERE clause filters rows; the HAVING clause filters **groups**.

```
sql
-- "Find only customers who have spent more than $1000."
SELECT C.customer_id, C.customer_name,
       SUM(OD.quantity * OD.unit_price) AS total_revenue
FROM ... -- (same joins as above)
GROUP BY C.customer_id, C.customer_name
HAVING SUM(OD.quantity * OD.unit_price) > 1000;
```

F. ORDER BY & LIMIT: Delivering Refined Results

Present your final answer clearly.

```
sql
-- "Who are our top 5 customers by revenue?"
SELECT ... -- (same as above)
ORDER BY total_revenue DESC
LIMIT 5;
```

Best Practices for Querying a Well-Modeled Database

1. **Start with the Data Model Diagram:** Before writing a line of code, trace the path you need. Identify your primary table and the joins required.
 2. **Use Table Aliases:** Always use short, clear aliases (e.g., Orders O, Customers C). It makes queries readable, especially with multiple joins.
 3. **Be Explicit with JOINS:** Prefer INNER JOIN and LEFT JOIN over old-style, comma-separated FROM clauses. They are clearer and safer.
 4. **Qualify Ambiguous Columns:** If a column name (like id or name) appears in more than one table, prefix it with the alias (O.order_id, C.customer_name).
 5. **Test Incrementally:** Build your query step-by-step.
 - o Start with a SELECT * FROM primary_table.
 - o Add one JOIN, check the results.
 - o Add columns to your SELECT.
 - o Add WHERE filters.
 - o Finally, add aggregation with GROUP BY.
 6. **Understand the Keys:** Use **primary keys** for uniqueness and **foreign keys** for your join conditions. This is the golden rule.
-

Common Pitfalls & How a Good Model Saves You

Pitfall	Problem in a Poor Model	How a Good Model Helps
Incorrect Aggregation(Cartesian Products)	Missing join conditions due to unclear relationships.	Foreign keys make the required ON clause obvious.
Redundant Data	Summaries are difficult because data is mixed with transactions.	Normalization separates entities, making GROUP BY logical and accurate.
Ambiguous Column Names	Multiple "Name" columns with no clear source.	Clear table names and aliases (Products.product_name vs. Suppliers.supplier_name).
Poor Performance	Table scans on unindexed, wide tables.	Indexes on primary and foreign keys make joins fast. Narrow, focused tables improve cache efficiency.

Demonstration: From Question to Query

Business Question: "We want to identify products that are frequently bought together in the same order to inform our product recommendation engine."

Thought Process with Our Model:

1. **Identify Core Tables:** The Order_Details table is the fact table linking Products to Orders.
2. **Need:** A self-join on Order_Details to match different products (product_A and product_B) that share the same order_id.
3. **Avoid:** Pairing a product with itself.
4. **Goal:** Count pair frequencies and list top pairs.

Resulting Query:

```
sql
SELECT
  P1.product_name AS product_A,
  P2.product_name AS product_B,
  COUNT(*) AS times_purchased_together
FROM Order_Details OD1
INNER JOIN Order_Details OD2
  ON OD1.order_id = OD2.order_id
  AND OD1.product_id < OD2.product_id -- Prevents duplicates (A,B) vs (B,A) and self-pairs
INNER JOIN Products P1 ON OD1.product_id = P1.product_id
INNER JOIN Products P2 ON OD2.product_id = P2.product_id
GROUP BY P1.product_id, P1.product_name, P2.product_id, P2.product_name
ORDER BY times_purchased_together DESC
LIMIT 10;
```

Conclusion: A well-modeled database transforms complex business questions into a series of logical, navigable steps in SQL. Your queries become an exercise in following the trails laid down by the data architect. The model ensures your answers are **correct, consistent, and performant**.

Section 3 :Normalization in Database Design

Normalization is the process of organizing data in a database so that each piece of information is stored in a clear, consistent, and non-redundant way. The main goal of normalization is not performance. The goal is **correctness, clarity, and long-term maintainability**.

When a database is well normalized, it becomes easier to:

- Avoid duplicate or conflicting data
- Update information safely

- Understand what each table represents
- Enforce business rules using the database itself

Normalization is especially important in transactional systems where data accuracy matters, such as payments, user accounts, or inventory.

Why Normalization Exists

In the real world, the same fact often appears in many places. For example, a user's email address may be referenced by orders, payments, and support tickets. If we store that email separately in every table, we create multiple copies of the same fact.

This leads to common problems:

- The same data is stored in multiple places
- Updates must be applied everywhere
- Some copies may become outdated
- The database no longer knows which value is correct

Normalization solves this by ensuring that **each fact lives in exactly one place**, and everything else refers to it.

Core Idea: One Fact, One Place

The central idea of normalization is simple:

Each table should represent one concept,
and each column should describe only that concept.

For example:

- A users table describes users
- An orders table describes orders
- An order_items table describes the relationship between orders and items

If a table starts describing multiple concepts at once, it is usually a sign that the design is not normalized.

Functional Dependency (Intuition, Not Math)

Normalization is based on the idea of **functional dependency**. In simple terms, this means:

If one value uniquely determines another value,
they should belong to the same table.

For example:

- user_id determines email and full_name
- order_id determines order_status and created_at
- item_id determines item_name and price

If a column depends on something other than the table's primary key, it likely belongs somewhere else.

First Normal Form (1NF): Atomic Values

First Normal Form requires that:

- Each column holds a single value
- There are no repeating groups or lists in a column

Bad example:

- A column called items = "Burger, Fries, Soda"

Good example:

- One row per item in an order_items table

1NF forces us to treat data as structured, not as text blobs. This is the foundation of all relational modeling.

Second Normal Form (2NF): No Partial Dependency

Second Normal Form applies when a table has a **composite primary key**.

A table is in 2NF if:

- It is already in 1NF
- Every non-key column depends on the whole primary key, not part of it

Example problem:

If a table has (order_id, item_id) as the key, then:

- quantity depends on both → correct
- item_name depends only on item_id → not correct

In this case, item_name belongs in the menu_items table, not in order_items.

Third Normal Form (3NF): No Transitive Dependency

Third Normal Form removes **indirect dependencies**.

A table is in 3NF if:

- It is in 2NF
- Non-key columns do not depend on other non-key columns

Example problem:

- orders table contains restaurant_id and restaurant_city

Here:

- order_id → restaurant_id
- restaurant_id → restaurant_city

This means restaurant_city does not depend directly on order_id. It belongs in the restaurants table.

3NF ensures that tables stay focused on their own entity and do not leak details of related entities.

What Normalization Gives You

A normalized database:

- Prevents update anomalies (changing data in many places)
- Prevents insertion anomalies (needing fake data to insert a row)
- Prevents deletion anomalies (losing important information accidentally)
- Makes schema evolution safer over time

These benefits matter most in systems where data lives for years and is modified by many different parts of the system.

Normalization Is a Starting Point, Not a Religion

Normalization is not the final goal of database design. It is the **starting point**.

We usually begin with a normalized schema because it:

- Captures the real-world structure accurately
- Makes rules explicit
- Reduces ambiguity

Later, when performance or access patterns demand it, we may choose to denormalize carefully. But without a solid normalized foundation, denormalization becomes risky and error-prone.

Key Takeaway

Normalization is about **designing for truth**.

Before optimizing for speed, scalability, or convenience, we first design a model that:

- Stores each fact once
- Clearly represents the real world
- Protects data from inconsistency

Normalization vs Denormalization

Normalization is the process of organizing data to minimize redundancy and dependency. In normalized schemas, each fact is stored in exactly one place. This improves consistency, simplifies updates, and reduces the risk of contradictory data. Highly normalized designs are common in transactional systems where correctness is critical.

Denormalization, on the other hand, intentionally introduces redundancy to improve performance or simplify queries. Instead of joining multiple tables at query time, related data may be stored together. This approach can dramatically reduce query complexity and latency, especially in read-heavy systems.

The choice between normalization and denormalization is not about right or wrong. It is a trade-off. Normalization favors data integrity and flexibility. Denormalization favors speed and simplicity at query time. Real-world systems often use a hybrid approach, starting with a normalized core model and selectively denormalizing where performance demands it.

Denormalization in Real-World Systems

Denormalization is the intentional introduction of redundancy into a database design in order to improve performance, simplify queries, or support specific access patterns.

Unlike poor design, denormalization is **a conscious and controlled choice**. It is usually applied *after* a system has a correct normalized model and clear evidence that performance or query complexity has become a problem.

Normalization optimizes for correctness and maintainability.
Denormalization optimizes for speed and simplicity at read time.

Both are tools. The key is knowing **when and why** to use each one.

Why Denormalization Exists

In normalized databases, answering a question often requires joining multiple tables. Joins are powerful, but they have a cost:

- They increase query complexity
- They consume CPU and memory
- They become expensive at scale
- They can slow down user-facing APIs

In systems where reads happen far more often than writes, repeatedly recomputing the same joins becomes inefficient. Denormalization avoids this by storing commonly needed data together, even if that data is duplicated.

The Core Trade-Off

Denormalization always trades one thing for another.

What you gain:

- Faster read queries
- Simpler SQL
- Fewer joins
- Better performance for dashboards and APIs

What you give up:

- Single source of truth
- Simpler writes
- Automatic consistency
- Easier updates

This trade-off must be intentional and justified by real usage patterns.

Example 1: Normalized vs Denormalized Orders

Normalized design (review)	Denormalized version: order_summary table
To show an order summary, we join multiple tables: <pre>SELECT o.order_id, u.full_name, r.name AS restaurant_name, SUM(oi.quantity * oi.unit_price_cents) AS total_cents FROM orders o JOIN users u ON u.user_id = o.user_id JOIN restaurants r ON r.restaurant_id = o.restaurant_id JOIN order_items oi ON oi.order_id = o.order_id GROUP BY o.order_id, u.full_name, r.name;</pre> This query is correct and flexible, but it runs joins every time.	<pre>CREATE TABLE order_summary (order_id BIGINT UNSIGNED NOT NULL, user_id BIGINT UNSIGNED NOT NULL, user_name VARCHAR(200) NOT NULL, restaurant_id BIGINT UNSIGNED NOT NULL, restaurant_name VARCHAR(200) NOT NULL, order_status ENUM('PLACED','PREPARING','DELIVERED','CANCELLED') NOT NULL, created_at DATETIME NOT NULL, total_cents INT UNSIGNED NOT NULL, item_count INT UNSIGNED NOT NULL, PRIMARY KEY (order_id), KEY idx_user_created (user_id, created_at), KEY idx_restaurant_created (restaurant_id, created_at)) ENGINE=InnoDB;</pre> Now the same question becomes: <pre>SELECT order_id, restaurant_name, total_cents, created_at FROM order_summary WHERE user_id = 1 ORDER BY created_at DESC LIMIT 10;</pre> This query is faster, simpler, and easier to maintain at the API layer.

Example 2: Duplicating Descriptive Fields

In a normalized model:

- restaurant_name lives only in the restaurants table

In a denormalized model:

- restaurant_name may also exist in orders or order_summary

Why do this?

Because restaurant names are frequently displayed and rarely change. The cost of occasionally updating them is lower than the cost of joining on every read.

This is a **low-risk denormalization**, and very common in production systems.

Example 3: Pre-Computed Aggregates

Another common form of denormalization is storing **pre-computed values**.

Instead of calculating totals every time:

`SUM(oi.quantity * oi.unit_price_cents)`

We store:

- total_cents
- item_count

These values are written once at order creation time and read thousands of times afterward.

This is especially useful for:

- Dashboards
 - Mobile APIs
 - Reporting pages
-

Keeping Denormalized Data in Sync

Denormalization moves complexity from reads to writes.

There are several common strategies to keep data consistent:

1. Application-level updates
When an order is created or updated, the application updates both normalized and denormalized tables.
2. Database triggers
The database automatically updates denormalized fields when source tables change.
3. Batch jobs
Denormalized tables are rebuilt periodically (hourly or daily).

Each approach has trade-offs, and the choice depends on system scale and consistency requirements.

When Denormalization Makes Sense

Denormalization is appropriate when:

- The system is read-heavy
- Query latency matters
- Data is displayed far more often than updated
- Access patterns are well understood
- Slight delays in consistency are acceptable

It is commonly used in:

- Analytics dashboards
 - Reporting databases
 - High-traffic APIs
 - Data warehouses
-

When Denormalization Is Dangerous

Denormalization becomes risky when:

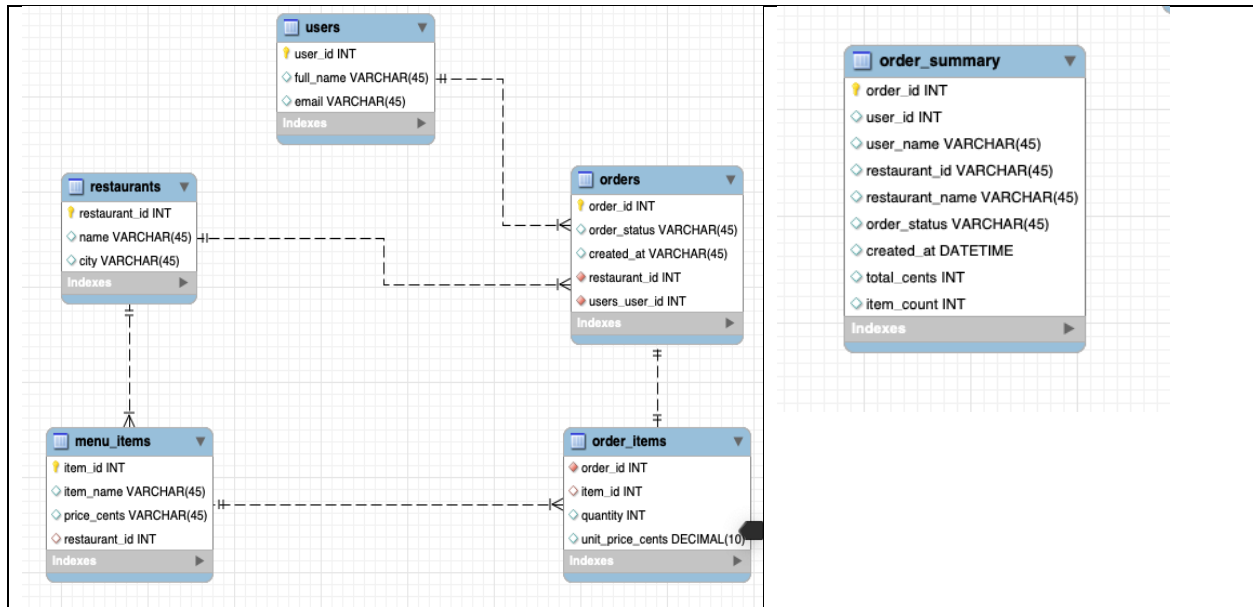
- Data changes frequently
- Strong consistency is required
- Business rules are complex
- The team lacks ownership over data updates

In these cases, redundancy can lead to silent data corruption and hard-to-debug errors.

Side-by-Side Diagram of Normalized vs Denormalized Design

Normalized Model (Source of Truth)	Denormalized Read Model
users	order_summary

----- user_id (PK) full_name email restaurants ----- restaurant_id (PK) name city orders ----- order_id (PK) user_id (FK) restaurant_id (FK) order_status created_at order_items ----- order_id (FK) item_id (FK) quantity unit_price_cents menu_items ----- item_id (PK) restaurant_id (FK) item_name price_cents	----- order_id (PK) user_id user_name restaurant_id restaurant_name order_status created_at total_cents item_count
Characteristics: • Each table represents one concept • No duplicated descriptive data • Requires joins to answer most questions • Strong consistency and data integrity	Characteristics: • Frequently accessed data stored together • Minimal joins • Faster reads • Data duplication is intentional



Normalization separates data to preserve truth.
Denormalization groups data to serve questions faster.

Exercise: “What Would You Denormalize and Why?”

You are building a food delivery system.
The product team wants:

- A fast “My Orders” page
- A restaurant dashboard
- A mobile API with low latency

You already have a fully normalized schema.

Exercise	Case	Expected Reasoning
Part A	From the following fields, choose which ones you would denormalize into an order_summary table: user_name user_email restaurant_name restaurant_city total_price item_count order_status	Good candidates: user_name (frequently displayed, rarely changes) restaurant_name (display-heavy, low update frequency) total_price (expensive to compute repeatedly) item_count (used in UI summaries) order_status (used constantly) Bad candidates:

	payment_method For each choice, explain why.	user_email (privacy, changes more often) restaurant_city (rarely needed per order) payment_method (business-sensitive, not always needed)
Part B (Design)	Look into the following sketch, then respond: • What happens if the restaurant name changes? • Who is responsible for keeping this table correct? <pre> order_summary (order_id, user_id, user_name, restaurant_name, total_price, item_count, created_at) </pre>	If the restaurant name changes, this table becomes incorrect because the old name is duplicated and stored per order, so past rows will not automatically update. That is a classic update anomaly. Responsibility for keeping it correct falls on the application or data pipeline that writes this table, not the database itself, since the data is denormalized and not enforced by foreign keys.

Key Takeaways

Denormalization is not bad design.
 It is **design with context**.

We normalize first to understand the data and protect correctness.
 We denormalize later to meet performance and usability needs.

Good engineers do not blindly follow rules.
 They understand the rules deeply, and then break them carefully.

Section 4: Designing for Read-Heavy vs Write-Heavy Systems

Not all systems behave the same under load. Some systems receive far more reads than writes, while others handle constant streams of updates and inserts.

Examples of Read-Heavy and Write-Heavy Systems

Not all systems are equally read-heavy and write-heavy. Every real system has a **dominant workload**, and good design starts by identifying it.

Read-Heavy Systems

In read-heavy systems:

- Data changes infrequently
 - The same data is read many times
 - Low latency is critical
-

Example 1: “My Orders” Page in a Food Delivery App

A typical user:

- Places 1 order per day (write)
- Opens the “My Orders” page many times (read)

Characteristics:

- Order data rarely changes after creation
- The UI repeatedly fetches the same information
- Slow responses harm user experience

Design choice:

- Use denormalized tables such as `order_summary`
- Store `restaurant_name`, `total_price`, and `item_count`

- Avoid repeated joins at read time

Why?

Because reads heavily outnumber writes.

Example 2: Restaurant Dashboard

A restaurant owner wants to see:

- Number of orders today
- Today's revenue
- Current order statuses

In one day:

- About 100 orders are written
- The dashboard may refresh hundreds or thousands of times

Design choice:

- Precomputed aggregates
- Summary tables
- Denormalization for fast reads

Example 3: User Profile Pages (Social / E-commerce Systems)

Displayed data:

- User name
- Profile photo
- Total orders
- Account status

These values:

- Are displayed very frequently
- Change rarely

Design choice:

- Duplicate selected fields across tables
- Optimize for fast reads and simple queries

This is a very common and safe form of denormalization.

Write-Heavy Systems

In write-heavy systems:

- Data is constantly being inserted
 - Write throughput matters more than read speed
 - Strong consistency is critical
-

Example 4: Logging Systems

Imagine:

- Each server writes hundreds of log entries per second
- Logs are mostly read only during debugging

Characteristics:

- Extremely high write volume
- Reads are infrequent and often batch-based

Design choice:

- Simple append-only tables
- Minimal indexing
- No complex joins or constraints

Why?

Indexes and joins slow down writes.

Example 5: Payment Systems

In payment processing:

- Every transaction is a critical write
- Errors are unacceptable
- Data integrity is mandatory

Characteristics:

- Writes must be atomic and accurate
- Reads are secondary

Design choice:

- Fully normalized schema
- Strong constraints and foreign keys
- Avoid risky denormalization

Here, correctness matters more than read performance.

Example 6: Event Ingestion and Telemetry Systems

Examples:

- User click events
- Sensor data
- Application telemetry

Characteristics:

- Millions of writes per day
- Analysis happens later

Design choice:

- Write-optimized schemas
 - Minimal validation at insert time
 - Aggregation performed by batch jobs or analytics pipelines
-

Quick Comparison

Read-Heavy Systems:

- Dashboards
- User profile pages
- Mobile APIs
- Analytics views

Write-Heavy Systems:

- Payments
 - Logs
 - Event ingestion
 - Telemetry pipelines
-

Key Takeaway

In read-heavy systems, we optimize schemas for fast queries.

In write-heavy systems, we optimize schemas for safe and fast inserts.

Or in simpler terms:

If the system reads more, we design for reading.

If the system writes more, we design for writing.

No real system is purely read-heavy or write-heavy.

One workload is usually dominant, and good design respects that reality.

Summary:

In **read-heavy systems**, such as dashboards, reporting platforms, or content delivery services, schema design often prioritizes fast retrieval. This may involve denormalized tables, pre-computed aggregates, or data structures optimized for common queries. The goal is to minimize joins and reduce query execution time.

In **write-heavy systems**, such as event ingestion pipelines or logging platforms, the primary concern is efficient and reliable writes. Schemas are often simpler, with fewer constraints and minimal indexing to avoid slowing down inserts. Data consistency may be enforced later through batch processing or downstream validation.

Understanding whether a system is read-heavy or write-heavy is essential. Designing a schema without this awareness can lead to bottlenecks, operational pain, and costly refactoring later.

Data modeling is not a one-time task. It evolves with the system, the business, and the scale of data. Strong models are not just technically correct; they are adaptable, understandable, and aligned with how data is actually used.

In this course, we will treat data models as living artifacts. We will learn how to design them thoughtfully, critique them realistically, and improve them as system requirements change.